



Algoritmos Recursivos

Estructura de Datos y Algoritmos I

Profesora: Elba Karen Sáenz García

Recursividad

Concepto fundamental en matemáticas y en computación.

En matemáticas se da el nombre de ***recursión*** a la técnica consistente en definir una función en términos de sí misma.

$$n! = n(n-1)(n-2)(n-3)(n-4)\dots(3)(2)(1)(0!)$$

$$n! = n(n-1)!$$

$$n! = n(n-1)(n-2)!$$



Recursividad

- Un **algoritmo recursivo** es un algoritmo que expresa la solución de un problema en términos de una llamada a sí mismo. La llamada a sí mismo se conoce como llamada recursiva o recurrente.



Características

Es una **alternativa** diferente **para implementar** estructuras de repetición ***(iteración)***.

Es un fenómeno que se presenta en muchos problemas de forma natural, delegando la **solución de un problema en la solución de otro más pequeño**

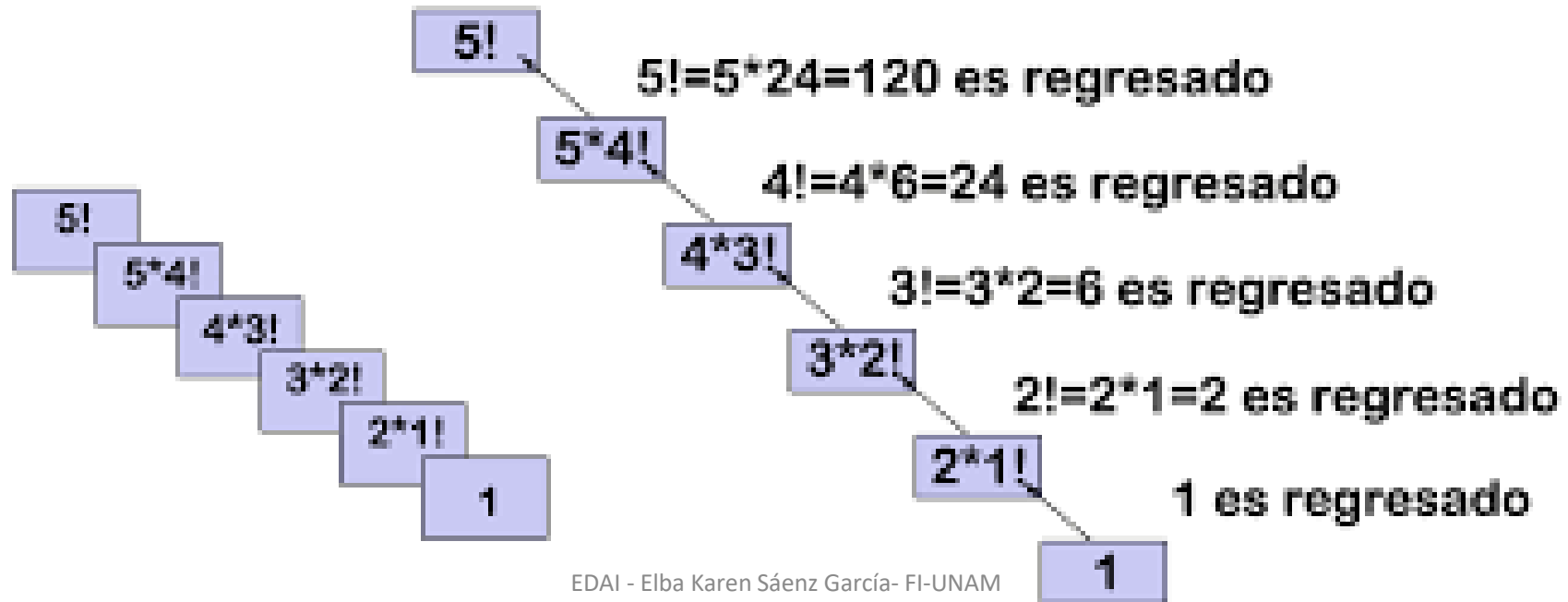
$$0!=1$$

$$1!=1$$

$$n!=n(n-1)(n-2)(n-3)(n-4)\dots(3)(2)(1)$$

$$5!=5*4!=5*(4*3!)=5*(4*(3*2!))=5*(4*(3*(2*1!)))=5*(4*(3*(2*(1*0!))))$$

Valor final: 20



Función recursiva

Se componen de:

- **Caso base:** una solución simple para un caso particular (puede haber más de un caso base).

No se requiere volver a llamar a la función

- **Caso Recursivo:** Una solución que involucra volver a utilizar la función original, con parámetros que se acercan más al caso base

Caso Recursivo

- El procedimiento se llama a sí mismo
- El problema se resuelve, tratando el mismo problema pero de tamaño menor
- La manera en la cual el tamaño del problema disminuye asegura que el caso base eventualmente se alcanzará

Ecuación de recurrencias

Una recurrencia es una ecuación o una desigualdad que describe una función en términos de su propio valor sobre entradas más pequeñas.

Ejemplo

$$f(n) = \begin{cases} nf(n-1) & \text{si } n > 0 \\ 1 & \text{si } n = 0 \end{cases}$$

$$n! = \begin{cases} n * (n-1)! & \text{si } n > 0 \\ 1 & \text{si } n = 0 \end{cases}$$

Algoritmo Factorial

Factorial (n)

Fin

Si $n \leq 1$

 retorna 1

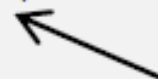
else

 retorna $n * \text{factorial}(n-1)$

Fin Si

Fin

```
long int factorial( long int n ) {  
    long int s1;  
    if (n <= 1)  
        return 1;  
    else {  
        s1= factorial(n-1);  
        return n*s1;  
    };  
};
```



La función se llama a sí misma.

Factorial (n)

Fin

Si $n \leq 1$

retorna 1

else

1) retorna $n * \text{factorial}(n -$

Fin Si

Fin

```
def fact(n):
```

```
    if n <= 1:
```

```
        return 1
```

```
    else:
```

```
        return n*fact(n-1)
```

```
def main():
```

```
    n=int(input('Número: '))
```

```
    print('El factorial es: {}'.format(fact(n)))
```

```
main()
```

- ¿Qué pasa si no hay caso base?

¿Por qué escribir programas recursivos?

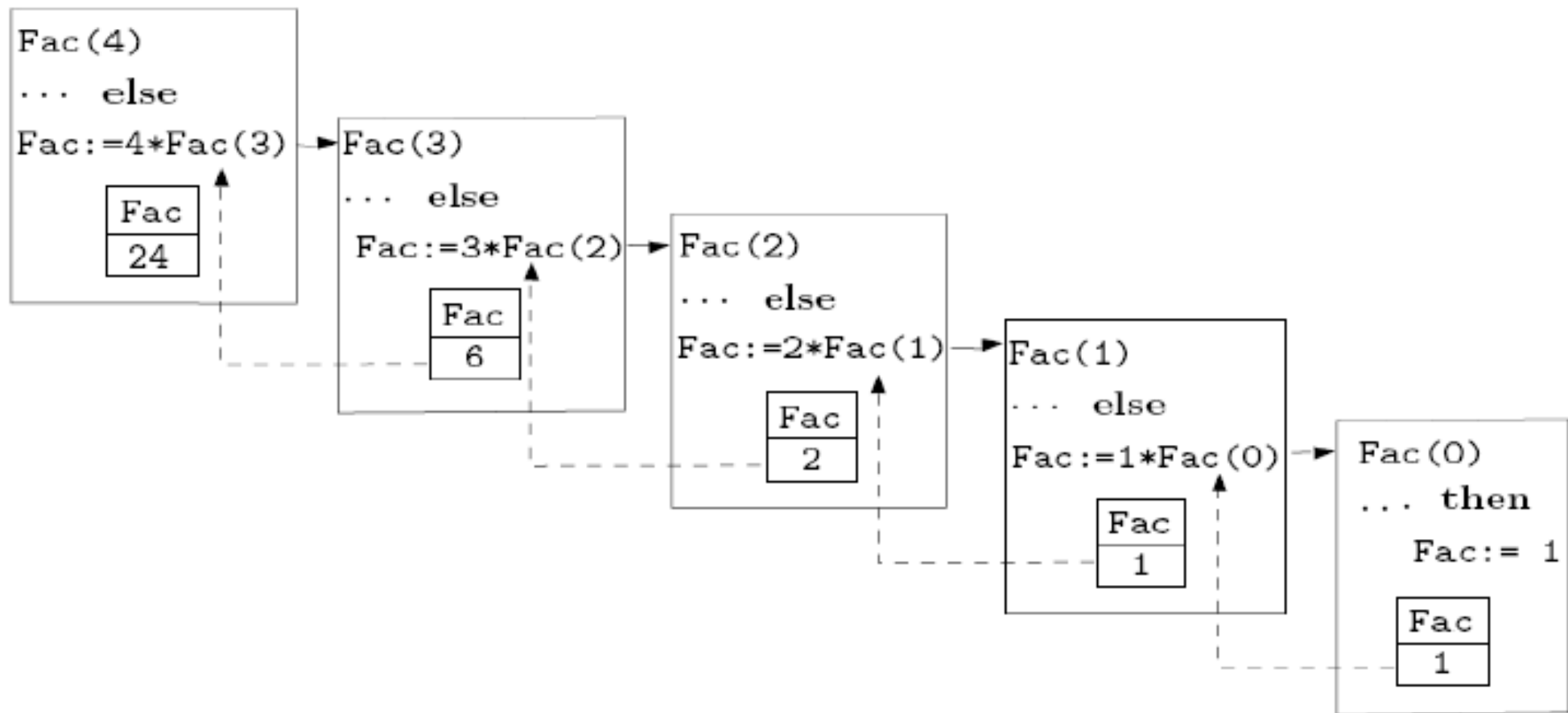
- Son mas cercanos a la descripción matemática.
- Generalmente mas fáciles de analizar
- Se adaptan mejor a las estructuras de datos recursivas.
- Los algoritmos recursivos ofrecen soluciones estructuradas, modulares y elegantemente simples.

Factibilidad y no factibilidad

- Es factible para simplificar código y cuando la estructura de datos es recursiva(P/E árboles)
- No es factible cuando los métodos usan arreglos grandes, cuando se cambia de manera impredecible.
- No es factible cuando las iteraciones son la mejor opción

Ventajas y Desventajas

- Ventajas:
 - Solucionan problemas recurrentes
 - Son algoritmos cortos
- Desventajas
 - Puede necesitar mucha memoria
 - Creación de muchas variables



Ejemplo

- Cálculo de los números de Fibonacci
- $F(0)=0$
- $F(1)=1$
- $F(2)=0+1=1$
- $F(3)=1+1=2$
- $F(4)=2+1=3$

.....

$$F(n)=F(n-1)+F(n-2)$$

¿Cuáles son los casos base?

$$F(n) = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F(n-1) + F(n-2) & \text{if } n > 1. \end{cases}$$

```
long fibonacci(int n) {  
  
long fn1, fn2, fn;  
int id;  
    if ( n == 0 || n == 1 )  
        return(n);  
        fn1 = fibonacci(n-1);  
        fn2 = fibonacci(n-2);  
        fn = fn1 + fn2;  
  
    return(fn);  
}
```

```
#include <stdio.h>  
long fibonacci(int n);  
  
main (){  
    int nthr=0;  
    int n;  
    long resul;  
    printf("\n Número a calcular? ");  
    scanf("%d", &n);  
    resul = fibonacci(n);  
    printf ("\nEl numero Fibonacci de %5d es %d", n, resul);  
}
```

Exponenciación Entera:

$$\begin{aligned} a^5 &= a^4 a = a^3 a a = a^2 a a a = a^1 a a a a \\ &= a^0 a a a a a = 1 a a a a a \end{aligned}$$

¿Cuál es el caso base?

Exponenciación Entera:

$$\bullet a^b = \begin{cases} a a^{b-1} & \text{si } b > 0 \\ 1 & \text{si } b = 0 \end{cases} \quad \begin{matrix} \text{caso base} \end{matrix}$$

Ecuación de Recurrencias

- $a^{10} = a^9 \cdot a = a^5 \cdot a^5$

- $$a^b = \begin{cases} a^{b/2} \cdot a^{b/2} & \text{si } b > 0 \text{ y } b \text{ es par} \\ a \cdot a^{b-1} & \text{si } b > 0 \text{ y } b \text{ es impar} \\ 1 & \text{si } b = 0 \text{ caso base} \end{cases}$$

Máximo común Divisor

- MCD de dos números enteros es el mayor número entero que los divide sin dejar residuo.
- Un número entero **d** se llama máximo común divisor de los números *a* y *b* cuando
 - *d* es divisor común de *a* y *b*
 - *d* es divisible por cualquier otro divisor común de los números *a* y *b*.

$$\bullet \text{ mcd} = \begin{cases} \text{mcd}(a - b, b) & \text{si } a \geq b \\ \text{mcd}(a, b - a) & \text{si } b > a \\ a & \text{si } b = 0 \text{ caso base} \\ b & \text{si } a = 0 \text{ caso base} \end{cases}$$

$$\text{mcd}(60, 48) = 12$$

Análisis de algoritmos recursivos

- Del algoritmo, código o pseudocódigo se obtiene una ecuación de recurrencias del tiempo de ejecución $T(n)$.

```
long int factorial( long int n ) {  
    long int s1;  
    if (n <= 1)  
        return 1;  
    else {  
        s1= factorial(n-1);  
        return n*s1;  
    };  
};
```



La función se llama a sí misma.

$$T(n) = \begin{cases} K_1 & n \leq 1 \\ T(n-1) + K_2 & n > 1 \end{cases}$$

Resolver la ecuación

Resolver Ecuación Recurrencias

- Encontrar una expresión no recursiva de T .
- Los tres métodos más utilizados para resolver ecuaciones de recurrencia son:
 - Inducción matemática,
 - Resolución de la ecuación característica
 - Solución por sustitución
- Otros

Solución de la ecuación por sustitución

- Solución de la
- ecuación por sustitución

$$T(n) = \begin{cases} K_1 & n \leq 1 \\ T(n-1) + K_2 & n > 0 \end{cases}$$

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

$$T(n) = T(n-1) + 1 \quad \dots \textcircled{A}$$

$$T(n-1) = T(n-2) + 1 \quad \dots \textcircled{B}$$

$\textcircled{B}$ en $\textcircled{A}$

$$T(n) = (T(n-2) + 1) + 1 = T(n-2) + 2$$

$$T(n) = T(n-2) + 2 \dots \textcircled{C}$$

$$T(n-2) = T(n-2-1) + 1 \dots \textcircled{D}$$

$$T(n) = T(n-3) + 1 + 2 = T(n-3) + 3$$

$$\vdots$$

$$T(n) = T(n-i) + i$$

En el caso base se cumple:

$$n-i=1 \Rightarrow T(n-i) = T(1)$$

$$i = n-1$$

$$T(n) = T(n - (n - 1)) + (n - 1)$$

$$T(n) = T(1) + n - 1$$

$$T(n) = 1 + n - 1 = n$$

$$\underline{T(n) = n} \quad \therefore \text{ es } \underline{O(n)}$$

Ejemplo 2

```
int Recursival( int n ) {  
    if (n <= 1)  
        return (1);  
    else  
        return ( 2*Recursival(n/2) );  
}
```

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n/2) + 1 & n > 1 \end{cases}$$

$$T(n) = T(n/2) + 1 \dots \textcircled{A}$$

$$T\left(\frac{n}{2}\right) = T\left(\frac{n/2}{2}\right) + 1 = T\left(\frac{n}{4}\right) + 1 \dots \textcircled{B}$$

$\textcircled{B}$ en $\textcircled{A}$

$$T(n) = T\left(\frac{n}{4}\right) + 1 + 1 = T\left(\frac{n}{4}\right) + 2 \dots \textcircled{C}$$

$$T\left(\frac{n}{4}\right) = T\left(\frac{n}{8}\right) + 1 \quad \text{D} \quad \text{D en C}$$

$$T(n) = T\left(\frac{n}{8}\right) + 1 + 2 = T\left(\frac{n}{8}\right) + 3 \quad \text{E}$$

$$T\left(\frac{n}{8}\right) = T\left(\frac{n}{16}\right) + 1$$

$$T(n) = T\left(\frac{n}{16}\right) + 1 + 3 = T\left(\frac{n}{16}\right) + 4$$

$$T(n) = T\left(\frac{n}{2^i}\right) + i$$

Para el caso base $\frac{n}{2^i} = 1$

$$T(1) = T\left(\frac{n}{2^i}\right) \quad n = 2^i \Rightarrow i = \log_2 n$$

$$T(n) = T(1) + \log_2 n$$

$$T(n) = 1 + \log_2 n$$

$$\Rightarrow O(\log n)$$

Ejercicio 3

```
int Rec3 (int n){  
    if (n <= 1)  
        return (1);  
    else  
        return ( Rec3(n-1) + Rec3(n-1) );  
}
```

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n-1) + 1 & n > 1 \end{cases}$$